01 - From a problem to a requirement

Read, predict and investigate

Name _____ Class _____ Date _____

Separate inputs, processing and outputs, then write a requirement you can test.

Textbook: Your first Python program: a problem before a program

Textbook: Designing solutions: investigate the problem

Visit the two textbook sections above. Use only the problem/requirement explanation; the book's separate game project is not this activity.

Worked example: badge order

A club sells badges for $3 each. Assume quantity is a non-negative whole number. For four badges: input 4; processing 4 * 3; output Total: $12. A testable requirement is: display quantity * 3 for any allowed quantity.

1A. Sort the facts

Statement	Input, processing, output or assumption?
The quantity is 5.	
Multiply the quantity by 3.	
Display Total: $15.	
The quantity must be a non-negative whole number.	

1B. Make the requirement testable

Replace 'the calculator should work well' with one observable requirement. Give a quantity and the exact expected total. Predict before running anything.

01 - From a problem to a requirement

Apply, compare and explain

Name _____ Class _____ Date _____

1C. Three requirement cards

For each card, identify input, processing, output and one assumption. These are paper/desktop scenarios, not instructions to operate equipment.

Card	Brief
A - cover order	Covers cost $2 each plus $4 for every order, including an order of zero.
B - simulated movement	A simulated marker begins at position 10. Read a non-negative distance, add it to the position and display the new position. No hardware is controlled.
C - status display	Read a device label and a supplied online Boolean. Display the label followed by online or offline. Use supplied data; do not scan a network.

1D. Check the boundary

For Card A, predict totals for 0 and 5 covers. Explain whether the handling charge applies to zero. Return to the textbook's requirement explanation if you need to justify the assumption.

Exit: show the reasoning

For five badges at $3 each, name input, processing and output. Write one requirement that a test could show to be wrong.

02 - Values, types and a saved program

Predict, save and run

Name _____ Class _____ Date _____

Run a saved sequence, distinguish text from numbers and predict simple calculations.

Textbook: Prepare your Python workspace
Textbook: Match a type to its meaning
All section links and timing guidance are on the activity page.

Read the named textbook examples. Keep the later optional test harness closed: no functions or loops are needed here. Use a school-approved Python 3 editor.

Worked sequence: badge cost

```
BADGE_PRICE = 3
quantity = int(input("Number of badges: "))
total = quantity * BADGE_PRICE
print(f"Total: ${total}")
```

Save as 02_badge_worked.py and run that saved file. Assume the entered text represents a non-negative whole number; handling unsuitable text comes later. For quantity 4, the expected output is Total: $12.

2A. Choose a representation

Meaning/value	Python type and reason
Identifier '0042'	
Count 4	
Measured length 2.5	
Paid True	

2B. Predict before checking

Predict: int('4') * 3; '4' * 3; 17 // 4; 17 % 4; 2 + 3 * 4; (2 + 3) * 4. Then check each in your editor/console.

02 - Values, types and a saved program

Make a small change and prove it

Name _____ Class _____ Date _____

2C. Change the saved program

Run the badge program for quantity 2. Change BADGE_PRICE to 4, save, and run the same file again. Record the expected and actual totals. If output did not change, check filename, saved version and which file ran.

Price / quantity	Expected	Actual after running
$3 / 2		
$4 / 2		

2D. Independent transfer: ticket cost

Tickets cost $4 each plus $1 per order, including zero-ticket orders. Starting with 02_ticket_starter.py or a blank file, write an input/calculation/output program. Assume valid non-negative whole-number input. Predict and run quantities 0, 1 and 3. You do not need selection or a loop.

Exit: explain a type choice

Why must input text be converted for this calculation? Why should identifier '0042' remain text? Optional extra: predict 2 ** 3, where ** means exponentiation.

03 - Requirements and a realistic schedule

Define what success means

Name _____ Class _____ Date _____

Translate a brief into measurable requirements and draw task durations with dependencies.

Textbook: Investigate the problem

Textbook: Plan work and dependencies

All section links and timing guidance are on the activity page.

Read the general requirements and scheduling sections. The textbook's separate Pygame game rules and workshop budget do not apply to this short planning activity.

Brief: a cover-order calculator

A club needs the cost of covers. Each costs $2 and every order has a $4 handling charge. Allowed quantities are non-negative whole numbers. Display the total clearly. No stock database, payment processing or live hardware is required.

3A. Investigate before coding

Write two measurable functional requirements, one assumption and one out-of-scope feature. Give one input with the independently calculated expected output.

3B. Read a schedule

Tasks must run in this order: A investigate (1 slot), B design (1 after A), C code (3 after B), D test (2 after C), E evaluate (1 after D). One slot is a planning unit, not necessarily one classroom lesson.

Task	1	2	3	4	5	6	7	8
A investigate	X							
B design		X						
C code			X	X	X			
D test						X	X	
E evaluate								X

03 - Requirements and a realistic schedule

Replan without hiding work

Name _____ Class _____ Date _____

3C. A predecessor takes longer

Coding now takes four slots. Redraw the chart, preserving every dependency and test/evaluation duration. State the finish slot and one defensible response to the original eight-slot deadline.

Task	1	2	3	4	5	6	7	8	9
A									
B									
C									
D									
E									

3D. Independent plan

Plan a desktop status display: investigate 1 slot, design 2, build 2, test 1, evaluate 1, all sequential. Draw or list the occupied slots and write one observable display requirement. Explain why removing testing is not the same as meeting the requirement sooner.

Exit

Why does a delayed predecessor affect later tasks? Distinguish a deadline, a duration and a dependency.

04 - Trace a sequence and represent a character

Follow the values in order

Name _____ Class _____ Date _____

Track changing values, distinguish an expected result from an observation, and convert one character code between bases.

Textbook: Trace and test

Textbook: Assignment updates a value

All section links and timing guidance are on the activity page.

Read the trace/assignment examples. Predict on paper before running 04_trace_worked.py. A name not yet assigned has no value in this trace; write not set rather than guessing zero.

```
quantity = 3
price = 2
handling = 4
subtotal = quantity * price
total = subtotal + handling
print(total)
```

4A. Complete the trace

After line	quantity	price	handling	subtotal	total / output
1					
2					
3					
4					
5					
6					

4B. Diagnose before changing

A pupil writes print("Total) with an unmatched quote. Is this a syntax fault or a wrong arithmetic rule? Repair the line to print the word Total. Then explain why a program that runs can still calculate the wrong amount.

04 - Trace a sequence and represent a character

Change the input, keep the reasoning

Name _____ Class _____ Date _____

4C. Fresh prediction

Keep price=2 and handling=4. Predict subtotal, total and output for quantity 0 and quantity 5. Run only after predicting; record actual output separately.

quantity	subtotal	Expected total/output	Actual after running
0			
5			

Worked representation: C

Character C has code point decimal 67. 67 = 64 + 2 + 1, so eight-bit binary is 01000011. Hexadecimal 43 means 4 * 16 + 3 = 67. The notation changes, not the code-point value. Encoding determines stored bytes.

4D. Independent character transfer

B has decimal code point 66. Write its eight-bit binary and hexadecimal code point and convert both back to decimal. Explain why one visible character need not use one UTF-8 byte.

Exit

State the output of the original six-line program and one reason an independent prediction is useful. Give B's binary and hex values without copying the C example.

05 - Conditions and Boolean reasoning

Read each condition precisely

Name _____ Class _____ Date _____

Evaluate comparisons and combine Boolean conditions with and, or and not.

Textbook: From an instruction to a decision

Textbook: Combine conditions and choices

Read the comparison/combined-condition examples. Python spells Boolean literals True and False. = assigns a value; == compares. The following battery rule is invented for a desktop exercise, not a real-flight operating rule.

Worked condition

```
battery = 29
authorised = True
ready = battery >= 30 and authorised
print(ready)
```

battery >= 30 is False, so ready is False even though authorised is True. Change values in 05_conditions_worked.py only after predicting.

5A. Complete a truth table

A	B	A and B	A or B	not A
False	False			
False	True			
True	False			
True	True			

5B. A comparison is not assignment

For score=7, evaluate score >= 5, score == 5 and score != 5. Explain why changing == to = is not an alternative comparison.

05 - Conditions and Boolean reasoning

Apply the rule to fresh cases

Name _____ Class _____ Date _____

5C. Both parts matter

battery	authorised	battery >= 30	ready
29	True		
30	False		
30	True		
31	True		

5D. Make grouping visible

Python evaluates not before and, and before or. Predict False or True and False, then (False or True) and False. Next predict True or False and False, then (True or False) and False. Use parentheses to communicate your intended grouping.

5E. Independent transfer

A club accepts entry only when age is at least 12 and member is True. Write the Boolean expression. Predict (age=11, member=True), (12,False) and (12,True). No input conversion or if statement is required yet.

Exit

Explain why one true part is insufficient for and. Give a pair of Boolean inputs that makes or true but and false.

06 - Choose exactly the intended branch

Predict which branch runs

Name _____ Class _____ Date _____

Use if/elif/else and trace a nested decision without executing both alternatives.

Textbook: Design before coding

Textbook: Implement the decision in Python

All section links and timing guidance are on the activity page.

Read the textbook's design, Python and trace examples. Indentation identifies a branch body. In one if/elif/else chain, the first matching branch runs and the remaining alternatives are skipped.

```
quantity = 0
if quantity < 0:
    result = "invalid"
elif quantity == 0:
    result = "empty"
else:
    result = "process"
print(result)
```

6A. Trace three inputs

quantity	Chosen branch	Printed result
-1		
0		
2		

6B. Implement the rule independently

Use 06_selection_starter.py or a blank file. An order total of at least $50 gets delivery charge 0; smaller non-negative totals get charge 5. Use an already-supplied numeric order_total, not interactive conversion. Test 49, 50 and 51.

06 - Choose exactly the intended branch

A decision inside a decision

Name _____ Class _____ Date _____

Worked nested decision

```
command = "START"
authorised = False
if command == "START":
    if authorised:
        print("ready")
    else:
        print("not authorised")
else:
    print("not a start request")
```

6C. Follow the actual path

Predict the output for the worked values, then for START/True and STOP/True. Mark which if each else belongs to. Check using 06_nested_worked.py after predicting.

6D. Independent transfer

A desktop display receives command='SHOW' and staff=False. If command is SHOW, staff users see 'full report' and others see 'summary'. Other commands produce 'unknown'. Write and test the nested decision for SHOW/False, SHOW/True and HIDE/True. It only prints a message.

Exit

Why can two separate if statements behave differently from an if/elif chain? Explain using the first matching branch.

07 - A reliable text command menu

Read a small input contract

Name _____ Class _____ Date _____

Normalise text deliberately and separate conversion, domain validation and permission.

Textbook: Combine conditions and choices

Textbook: Accept only usable input

All section links and timing guidance are on the activity page.

Use the named textbook paragraphs for validation, not the later file/exception-handling lab. This menu only prints a response. It does not control a drone, start a process or send network requests.

Contract

Accept START, STOP or STATUS, ignoring letter case and whitespace at the ends. START prints Start requested; STOP prints Stop requested; STATUS prints Status requested. Any other input, including blank input, prints Unknown command. Internal spaces are not removed.

Worked normalisation

```
raw = "  status  "
command = raw.strip().upper()
print(command)  # STATUS
```

strip() removes leading/trailing whitespace; upper() returns uppercase text. Neither proves that the resulting command is allowed. Normalisation follows this explicit contract; it is not a general instruction to strip or change all user data.

7A. Predict the normalised text and response

Raw input	Normalised text	Expected response
' start '		
'Stop'		
'STATUS'		
' '		
'ST ART'		

07 - A reliable text command menu

Implement, test and explain a limit

Name _____ Class _____ Date _____

7B. Complete the menu

Use 07_menu_starter.py. Preserve raw input, create command = raw.strip().upper(), then write if/elif/else branches satisfying the contract. No loop, function or try/except is required. Predict at least one valid, one blank and one unknown input before running.

7C. Transfer the rule carefully

A separate display accepts an already-supplied integer rating from 1 to 5 inclusive. Write a Boolean range expression. Why does int('-4') succeeding not mean -4 is an allowed rating? What happens if int('three') is attempted? Explain without implementing exception handling.

7D. A normalised command is not permission

Someone enters ' start ' but is not authorised to use a real service. Does this menu's acceptance prove permission? Explain the difference using the nested decision from Lesson 6.

Exit

Name one input accepted after normalisation and one still rejected. Explain why input conversion and an allowed-value check are different.

08 - Test both sides of the boundary

A typical value is not enough

Name _____ Class _____ Date _____

Predict discriminating tests, find a boundary defect and retain failure/repair/retest evidence.

Textbook: Trace a decision

Textbook: Test the boundary

All section links and timing guidance are on the activity page.

Read the boundary example, then state the expected rule before looking at code. Allowed ratings are integers from 1 to 5 inclusive. In this exercise rating is already an integer; input conversion is outside the tested contract.

8A. Predict from the requirement

rating	Expected accepted?	Why this case?
0		
1		
3		
5		
6		

8B. Inspect the faulty program

```
rating = 3
accepted = rating > 1 and rating < 5
print(accepted)
```

Without running it yet, predict its result for all five ratings. Which inputs distinguish the faulty rule from the requirement? Why might testing only 3 miss the defect?

08 - Test both sides of the boundary

Repair, retest and transfer

Name _____ Class _____ Date _____

8C. Retain actual evidence

Use 08_boundary_fault.py, changing rating for each run. Record actual results, repair the condition, and rerun the same cases. Keep the original file or a clearly identified copy; do not replace earlier evidence with the repaired outcome.

rating	Expected	Actual before	Actual after repair
0			
1			
3			
5			
6			

8D. Independent transfer

A second requirement accepts order quantities from 2 to 8 inclusive. Write the condition and choose tests just outside/inside both endpoints plus a typical value. Predict them independently, then run. Do not simply change expected answers to match your program.

Exit

For the original 1-5 rule, name four cases that distinguish both boundaries and give their expected results. Explain one changed test you would try after a later edit.